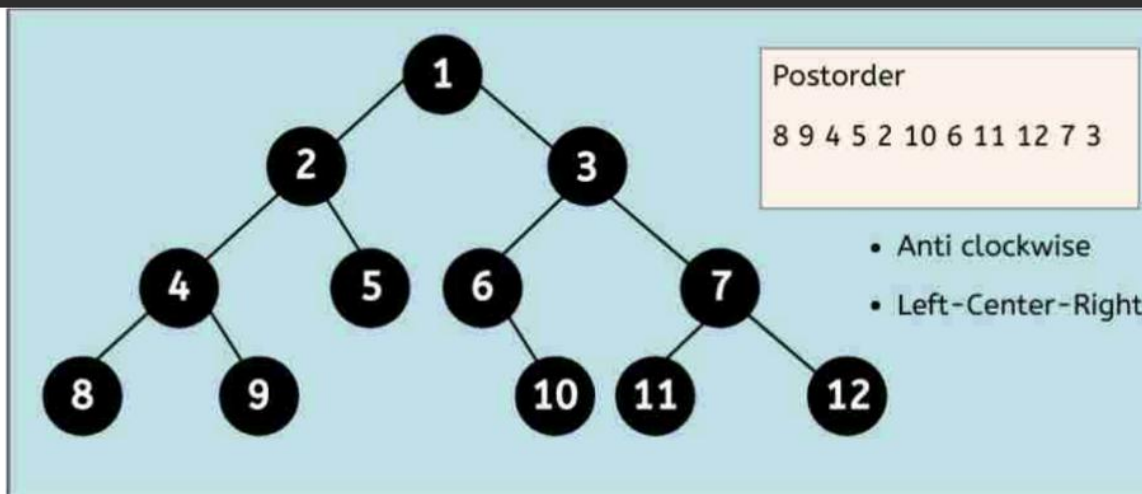


Q.No:4



Constraints

$T \leq 100$
 $1 \leq n \leq 1000$
 $0 \leq k < n$

Input Format

The first line of input consists of the number of test cases T . Then T lines follow each describing a test case. The first line of each test case consists of a number n . Then follow n lines each describing the friendship of a student. Each line starts with a number k telling the number of friends the i th person has. Then follow k integers separated by space which tells the index (1 based) of his friends. All these integers on the i th line are greater than i itself. (See sample for clarity).

Output Format

For each test case, if it is possible to divide the class into teams, then print a line of L's and R's denoting which team that student is a part of. If it is impossible to divide the class then print -1 instead. If there are multiple solutions print the one that would come first in the dictionary.

INPUT (STDIN)

```
7
90 70 50 75 130 110 150
```

EXPECTED OUTPUT

```
50 75 70 110 150 130 90
```

INPUT (STDIN)

```
9
50 23 56 89 43 75 130 110 150
```

EXPECTED OUTPUT

```
43 23 75 110 150 130 89 56 50
```

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
struct node *left,*right;
```

Test Case 3

KEYWORD

```
struct node* newNode(int item)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
```

```
int data;
struct node *left;
struct node *right;
};

struct node* newNode(int value)
{
    struct node *temp;
    temp = (struct node*)malloc(sizeof(struct node));
    temp->data = value;
    temp->left = NULL;
    temp->right = NULL;

    return temp;
}

struct node* insert(struct node *root, int value)
{
    if (root == NULL)
    {
        return newNode(value);
    }

    if (value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else
    {
        root->right = insert(root->right, value);
    }

    return root;
}

void postorder(struct node *root)
{
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

int main()
{
    int n;
    int a[1000];

    struct node *root = NULL;
```

```
scanf("%d", &n);

for (int i = 0; i < n; i++)
{
    scanf("%d", &a[i]);
}
for (int i = 0; i < n; i++)
{
    root = insert(root, a[i]);
}
printf("output:\n");
postorder(root);

return 0;
}
```

Output:

Test_case 1:

```
7
90 70 50 75 130 110 150
output:
50 75 70 110 150 130 90
```

Test_case 2:

```
9
50 89 56 23 75 130 110 150 43
output:
43 23 75 56 110 150 130 89 50
```